



UNIVERSIDAD AUTONOMA METROPOLITANA

DIVISIÓN CBI

Física Computacional

**Paralelización de algoritmo de Velocity Verle en CUDA desde un código
de c++ asistido por Gemini**

Alumno: AVALOS CAMPECH OCTAVIO

12/12/2025

1. Contexto y Origen del Código

Este proyecto fue desarrollado en el entorno de Google Colab utilizando el modelo Gemini como herramienta principal para la conversión y optimización de código. Se partió de un código base de Dinámica Molecular escrito en C++ (ejecución en CPU) y se utilizaron *prompts* iterativos para reescribir la parte más intensiva computacionalmente, el cálculo de fuerzas, utilizando la arquitectura de paralelismo masivo de CUDA (GPU).

Objetivo del código: Simular la evolución de un sistema de partículas (modelo de gas o líquido de Lennard-Jones) bajo condiciones de frontera periódicas (PBC), utilizando el algoritmo de integración de Verlet para la dinámica temporal.

2. Descripción Funcional del Código C++ (Host - CPU)

El código se compone de tres archivos principales: Atomo.h, Atomo.cpp y main.cpp, que definen la clase Simulacion y la lógica de alto nivel.

Archivo	Responsabilidad Principal
Atomo.h	Declaración de la clase Simulacion, variables del sistema ($N_A, L, \Delta t$), y la interfaz de la función paralela CalcularFuerzas_CUDA.
Atomo.cpp	Implementación de la lógica MD en el Host (CPU): Inicialización, integración de Verlet (PasoDeVerlet), cálculos termodinámicos, y el <i>wrapper</i> que llama al código de la GPU (CalcularFuerzas).
main.cpp	Punto de entrada del programa. Crea una instancia de Simulacion y llama a la función Simular().

3. Implementación Paralela en CUDA (Device - GPU)

La porción paralela se encuentra en el archivo cuda_md.cu y se enfoca exclusivamente en la tarea más costosa de la simulación: calcular la interacción de potencial de Lennard-Jones y las fuerzas entre cada par de partículas.

3.1. Algoritmo de Fuerza (Fuerza Bruta / *Brute Force*)

El primer código implementó el método de la fuerza bruta (*Brute Force*).

Lógica central: El núcleo de CUDA (kernel) asigna un hilo de ejecución a cada par único de átomos, o más comúnmente, a cada átomo (i), y luego ese hilo itera sobre todos los demás átomos (j) en el sistema.

- Asignación de Hilos: Cada hilo se identifica con un índice global único (idx). Este índice se mapea al índice de un átomo i (generalmente $i = \text{idx}$).
- Cálculo de Pares: Cada hilo i recorre todos los átomos j (donde $j > i$ para evitar doble conteo y auto-interacción).
- Función de Potencial/Fuerza: Para cada par (i, j), se calcula:
 - La distancia mínima entre i y j, aplicando las condiciones de frontera periódicas (PBC).
 - La energía potencial de Lennard-Jones (LJ): $V_{\{LJ\}}(r) = 4\epsilon \left[\left(\frac{\sigma}{r} \right)^{12} - \left(\frac{\sigma}{r} \right)^6 \right]$.
 - La fuerza resultante, $F = -\nabla V_{\{LJ\}}$.

3.2. Mecanismos de Paralelismo en cuda_md.cu

El proceso se puede resumir en los siguientes pasos de transferencia de datos y ejecución:

1. Transferencia Host a Device (H → D):

- Las posiciones de los átomos (p_{host}) y las variables del sistema (L, r_c^2, N_A, N_D) se copian de la memoria principal (RAM) a la memoria global de la GPU (VRAM).

2. Lanzamiento del Kernel:

- El Host lanza el kernel CUDA. Típicamente, el número de bloques y de hilos se configura para tener un hilo por átomo (N_A).

3. Ejecución del Kernel (CalcularFuerzas_CUDA):

- Cada hilo i lee su posición y calcula la interacción con todos los otros átomos j.

- Se utiliza un mecanismo de reducción atómica (por ejemplo, `atomicAdd`) para acumular las fuerzas F_i y la energía potencial total U_{total} . Esto es crucial porque múltiples hilos acceden y modifican la misma ubicación de memoria para la fuerza F_i y U_{total} .

4. Transferencia Device a Host (D $\rightarrow$ H):

- Una vez que todos los hilos han terminado, las fuerzas acumuladas (f_{device}) y la energía potencial total ($u_{\text{potencial_device}}$) se copian de vuelta a la CPU para ser utilizadas en el paso de integración de Verlet.

4. Eficiencia y Limitaciones (El motivo de las Cell Lists)

El método de fuerza bruta es conceptualmente simple de paralelizar, pero su rendimiento se ve limitado por su complejidad temporal.

- Complejidad: El cálculo de pares tiene una complejidad $O(N_A^2)$ (cuadrática), donde N_A es el número de átomos.
 - Para $N_A = 1000$ átomos, el número de cálculos de pares es approx 500,000.
 - Para $N_A = 100,000$ átomos, el número de cálculos es approx 5,000,000,000.
- Escalabilidad: Aunque el paralelismo de CUDA mitiga el tiempo por par, la complejidad cuadrática hace que el tiempo de simulación crezca rápidamente a medida que aumenta el tamaño del sistema.
- Overhead: Gran parte del cálculo de fuerza es desperdiciado, ya que las interacciones de Lennard-Jones se anulan más allá del radio de corte (r_c).

Conclusión: Este primer código en CUDA demostró la capacidad de paralelizar una tarea $O(N^2)$ en la GPU, resultando en una aceleración considerable respecto a la CPU para sistemas pequeños a medianos. Sin embargo, su limitación de escalabilidad fue la razón directa por la que posteriormente se migró

a la implementación con Cell Lists (complejidad $O(N_A)$), lo cual permite escalar a sistemas mucho más grandes y realistas.

Nota final: Este código fue exitosamente convertido y depurado con la asistencia de Gemini, basándose en la necesidad de optimizar la rutina de cálculo de fuerzas del código base en C++.